

0.0.1. Pourquoi écrire « #include » ?

Une fonction de la console prend de la place dans la cartouche. On écrit « #include <nom> » pour dire qu'on s'en sert : sans la ligne, le compilateur la refuse ; avec, il la grave — et seulement si le programme l'emploie vraiment.

1. Ce qu'il faut comprendre

Au 0.0, le programme ne faisait rien, et il n'avait aucune ligne #include. À partir de maintenant, presque chaque programme commence par quelques lignes #include <...>. Cette étape explique pourquoi, une fois pour toutes, avant d'en écrire une.

La cartouche est petite. Une cartouche Game Boy, c'est **32 Ko** : 32 768 octets pour tout le jeu — le programme, les dessins, la musique. Un octet est une case qui retient un nombre de 0 à 255. Chaque instruction du programme en occupe quelques-unes.

Une fonction de la console prend de la place. Écrire un mot (texte()), lire la manette (bouton()), jouer une note (note()) : derrière chacune, il y a du code, des petites routines, des tables (les dessins des lettres, les hauteurs des notes). Tout cela doit être **gravé dans la cartouche** pour que la console puisse s'en servir.

Rien n'est là d'office. Si la console mettait toutes ses fonctions dans chaque cartouche, un programme qui n'écrit qu'une lettre emporterait aussi la musique, les lutins, la police entière... des milliers d'octets pour rien. La règle est donc celle du langage C : **ce qu'on veut employer, on l'inclut.**

La ligne #include <texte> se lit : « ce programme se sert de texte() ». Le **nom entre les chevrons < >** est celui de la fonction, écrit **exactement** comme dans le programme : #include <texte> pour texte(), #include <ALPHABET> pour ALPHABET, #include <Tuile> pour une Tuile. Le # au début dit que ce n'est pas une instruction du jeu : c'est une indication donnée au compilateur, avant la compilation.

Sans la ligne, le compilateur refuse, et il dit laquelle écrire : « il faut « #include <texte> » pour employer texte() ». Personne n'a à deviner. Dans l'atelier, un bouton l'écrit pour toi.

Avec la ligne, il ne grave que ce qui sert. Une ligne #include de trop ne coûte **rien** : si le programme n'appelle jamais la fonction, rien n'est gravé. Le panneau ROM de l'atelier dit, pour chaque ligne, si elle sert et combien d'octets elle coûte.

Ce qui ne s'inclut pas : image(), images(), retard(), ms() et secondes() sont **natives** — elles font tourner la boucle du jeu et ne coûtent rien. C'est pourquoi le 0.0 n'avait besoin d'aucun #include.

La suite du parcours avance une ligne à la fois. Chaque fois qu'une fonction de la console sert pour la première fois, une étape la présente seule — sa ligne #include, ses arguments, ce qu'elle coûte —, juste avant la leçon qui l'emploie. Tu n'as jamais deux lignes nouvelles à comprendre en même temps.

Essaie : efface la ligne #include <texte>, puis lance : lis le message du compilateur. Remets-la : tout repart.

Les fonctions de cette leçon — et le tuto de chacune

- #include <texte> → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **474 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         18 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   94 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        208 o ← CopierTuiles – 13 lettres sur 43
"INCLUS DONC GRA..." 17 o ← texte() – un texte gravé
```

Lettres gravées : ACDEGILNORSUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- Pourquoi écrire #include ? ----
// La ligne ci-dessous dit au compilateur : « ce programme se sert de texte() ».
//
// #include <texte>
// |           |
// |           +-- le nom de la fonction, écrit comme dans le programme
// +----- « # » : une indication pour le compilateur, pas une instruction du jeu
//
// Sans elle, texte() est refusée : une fonction de la console prend de la
// place dans la cartouche, et ne s'emploie qu'inclus. Avec elle, texte() est
// gravée – et seulement parce que le programme l'appelle vraiment.

#include <texte> // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(1, 8, "INCLUS DONC GRAVE"); // texte() existe : sa ligne #include est là

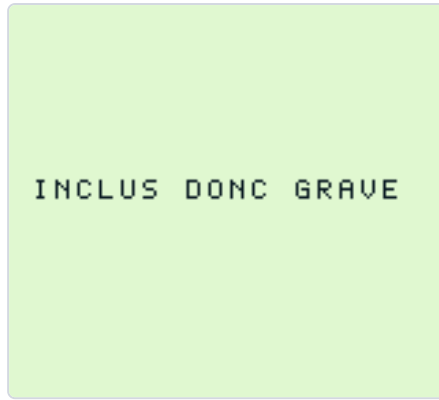
    while (true) { // la boucle du jeu
        image();   // image() est native : elle ne s'inclut pas
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

INCLUS DONC GRAVE écrit au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le texte est écrit : `texte()` est bien incluse

Ensuite : **0.0.2. La fonction `texte()` — écrire un mot** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++